Convolutional Neural Networks

How Machines Learn to See

Arjo Kar • Lamisa Zahin • Zannatun Nayeem Ohee

The Story Behind CNN

The Problem
Naive Doctor
The Insight
CNN Solution

Step 1 of 4 – The Problem

A hospital.

10,000 X-rays.

Patterns hidden across thousands of patients.

Who can find them?

The Story Behind CNN

The Problem

Naive Doctor

The Insight

CNN Solution

Step 2 of 4 – The Naive Doctor

Sees scan 1.

✗ Forgets it.

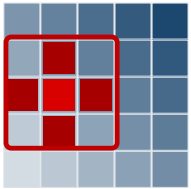
Sees scan 2.

✗ Forgets it.

No memory.

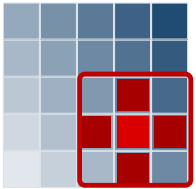
No pattern.

Scan 1



✗

Scan 2



same pattern – different position – completely missed.

The Story Behind CNN

The Problem
Naive Doctor
The Insight
CNN Solution

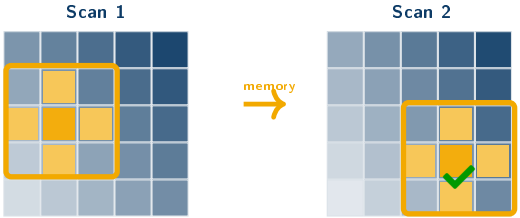
Step 3 of 4 – The Insight

Experienced doctor sees scan 1.

✔ Remembers the pattern.

Scan 2 matches.

Recognised!



✔ Same cross pattern – different position – recognised instantly.

The Story Behind CNN

The Problem

Naive Doctor

The Insight

CNN Solution

Step 4 of 4 – CNN – The Experienced Doctor

- ✔ Trained on thousands of scans.
- ✔ Remembers every pattern it has ever seen.
- ✔ Spots danger instantly – every single time.

That is CNN.

Human Vision vs Computer Vision

Human Sees
Computer Sees
The Gap
CNN Bridges It

Step 1 of 4 – What a Human Sees

You glance at a photo.

Instantly you see a cat.

Meaning.
Context.
Object.

"A Cat!"

Human Vision vs Computer Vision

Human Sees

Computer Sees

The Gap

CNN Bridges It

Step 2 of 4 – What a Computer Sees

The same photo.

Just a grid of numbers.

0 to 255.

No cat.

No meaning.

"Numbers"

same image – to the machine, only a matrix of numbers.

Human Vision vs Computer Vision

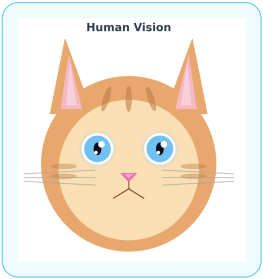
Human Sees

Computer Sees

The Gap

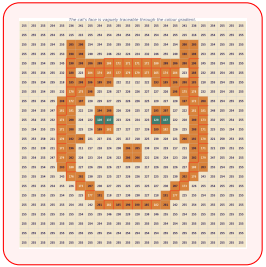
CNN Bridges It

Step 3 of 4 – The Gap



Sees: "Cat"

THE GAP



Sees: "Numbers"

How do we close this gap?

Human Vision vs Computer Vision

Human Sees

Computer Sees

The Gap

CNN Bridges It

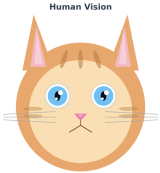
Step 4 of 4 – CNN Bridges the Gap

Numbers in

CNN

filters

Meaning out



Cat: 97%

✔ Numbers in. Meaning out. That is CNN.

What is a Neural Network?

What is a NN?

Problem 1

Problem 2

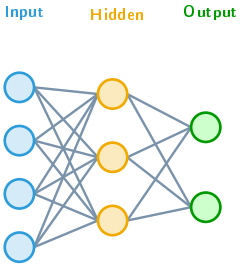
Problem 3

Before CNN – What is a Neural Network?

Inspired by the **human brain**.

Layers of **neurons** that pass signals forward.

Learns by adjusting weights from examples.



weights on every connection

Problems with Neural Networks

What is a NN?

✕ Problem 1

✕ Problem 2

✕ Problem 3

✕ **But – Normal NN Struggles with Images**

Works well for
simple data.

But images are
huge, spatial,
and complex.

3 big problems arise.

Problem 1 – Way Too Many Parameters

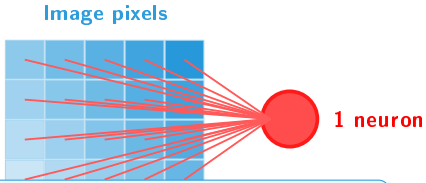
✕ Problem 1

✕ Problem 2

✕ Problem 3

✕ Problem 1 – Every pixel connected to every neuron

In a normal neural network, **every single pixel** is connected to **every single neuron** – each connection has its own weight. *Start simple: one pixel grid, one neuron. How many weights?*



$5 \times 4 = 20$ pixels $\rightarrow$
20 weights for just 1 neuron.
Now imagine the real image...

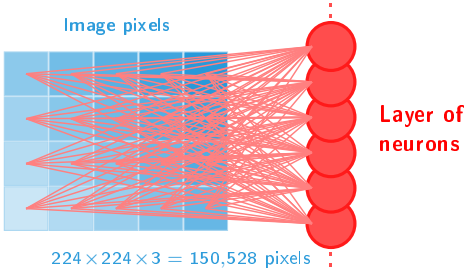
Problem 1 – Way Too Many Parameters

✗ Problem 1

✗ Problem 2

✗ Problem 3

✗ **Problem 1 – Every pixel to EVERY neuron**



$150{,}528 \times 1{,}000 =$ **150 million weights!**

Problem 1 – Way Too Many Parameters

✗ Problem 1

✗ Problem 2

✗ Problem 3

✗ Problem 1 – Why This Breaks Everything

A normal neural network **connects every pixel to every neuron**.

For a 224×224 colour image:

$224 \times 224 \times 3 = 150{,}528$ inputs

$\times$ just 1000 neurons

= 150 million weights.

In one layer alone.

Impossible to train efficiently.

Overfits. Slow. Expensive.

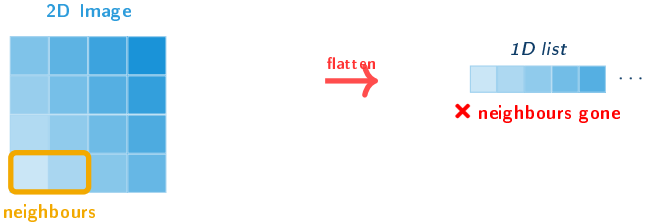
Problem 2 – Spatial Structure is Destroyed

✕ Problem 1

✕ **Problem 2**

✕ Problem 3

✕ Problem 2 – What Flattening Does



All spatial relationships vanish.
No idea which pixels were neighbours.

Problem 2 – Spatial Structure is Destroyed

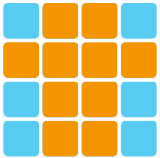
✕ Problem 1

✕ **Problem 2**

✕ Problem 3

Real-Life Analogy

Assembled Puzzle



✓ clear shape

flatten →

Random Pile

✕ same parts, wrong order

That is exactly what flattening does.

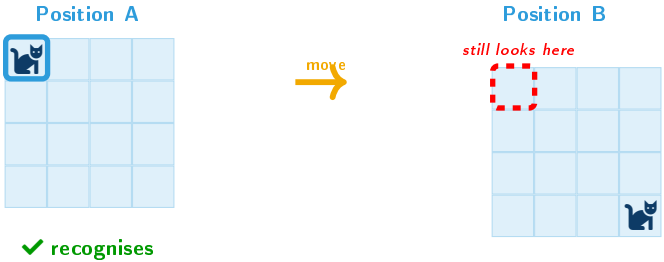
Problem 3 – No Translation Invariance

✕ Problem 1

✕ Problem 2

✕ Problem 3

✕ Problem 3 – Object Moves – Network Fails



The cat shifted position – the network **still looks at the old spot**.
It must **relearn from scratch** that the same cat exists at every new position.
✕ **Every shift = a brand new learning problem.**

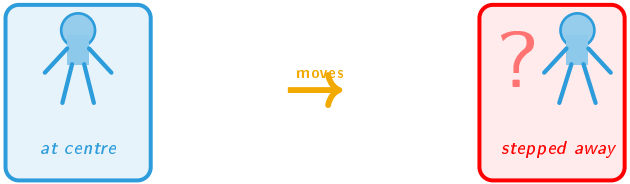
Problem 3 – No Translation Invariance

✕ Problem 1

✕ Problem 2

✕ **Problem 3**

Real-Life Analogy



✔ **recognised**

✕ **who is that?**

A normal NN knows the person
only when he stands in the exact same spot.

How CNN Solves All Three Problems

✗ What went wrong	✓ How CNN fixes it
1. Parameter explosion	**Local Connectivity** Each neuron sees only a **small patch**. Millions of weights → just hundreds.
2. Spatial structure lost	**Shared Weights** The same filter slides across the image – spatial layout preserved, weights reused.
3. No translation invariance	**Pooling** Summarises local regions – network becomes robust to shifts in object position.

How CNN Solves All Three Problems

✗ What went wrong	✓ How CNN fixes it
1. Parameter explosion →	**✓ Local Connectivity** Each neuron sees only a small patch. Millions of weights → just hundreds.
2. Spatial structure lost →	**Shared Weights** The **same filter** slides across the image – spatial layout preserved, weights reused.
3. No translation invariance →	**Pooling** Summarises local regions – network becomes robust to shifts in object position.

How CNN Solves All Three Problems

✗ What went wrong		✓ How CNN fixes it
1. Parameter explosion	→	**✓ Local Connectivity** Each neuron sees only a small patch. Millions of weights → just hundreds.
2. Spatial structure lost	→	**✓ Shared Weights** The same filter slides across the image – spatial layout preserved, weights reused.
3. No translation invariance	→	**Pooling** Summarises local regions – network becomes **robust to shifts** in object position.

Building Blocks of CNN – Conv Layer

Input Image → **Conv Layer** → ReLU → Pooling → FC Layer

Conv Layer – Detect local patterns

Slides small filters over the image and detects features like edges/textures.

Output: feature map of pattern responses.

Real-life analogy: like a magnifying glass scanning for a known pattern.

Building Blocks of CNN – ReLU

Input Image → Conv Layer → **ReLU** → Pooling → FC Layer

ReLU – Rectified Linear Unit

ReLU (**Rectified Linear Unit**) is a standard activation stage placed between convolutional transformations.

Role in pipeline: helps the network build richer representations.

Real-life analogy: like a checkpoint between processing stages.

Building Blocks of CNN – Pooling

Input Image → Conv Layer → ReLU → **Pooling** → FC Layer

Pooling – Keep strongest responses

Downsamples local patches by keeping the strongest activation (max pool).

Output: smaller map with key signals preserved.

Real-life analogy: like skimming a page and retaining only key points.

Building Blocks of CNN – FC Layer

Input Image → Conv Layer → ReLU → Pooling → **FC Layer**

FC Layer – Final decision layer

Combines extracted features and computes class scores/probabilities.

Output: final prediction (e.g., Cat 94%).

Real-life analogy: like a jury hearing evidence and giving one verdict.

Building Blocks of CNN – All Together

Input Image → Conv Layer → ReLU → Pooling → FC Layer

Full flow: features are extracted, transformed, condensed, then combined for the final prediction.

Real-life analogy: from observation → interpretation → summary → final decision.

THE CONVOLUTION LAYER

How does a CNN process an image?

Everything starts with one operation: **Convolution**

What is Convolution?

1D (Mathematics)

$$(a * b)_t = \sum_{\tau} a_\tau \, b_{t-\tau}$$

Slide one signal over another, multiply & sum.

2D (CNN)

$$O[i,j] = \sum_m \sum_n I[i+m, j+n] \cdot K[m,n]$$

I = input image $\quad$ K = kernel

Key intuition

Slide a small pattern (kernel) over the input.

At each position: multiply element-wise, sum to one number.

Repeat everywhere $\Rightarrow$ **feature map**.

The Kernel

What is a Kernel?

A small matrix of weights that slides across the input.

- Detects a specific pattern (edge, texture...)
- Different kernels $\rightarrow$ different features
- In CNNs, kernels are **learned** from data

The Kernel

What is a Kernel?

A small matrix of weights that slides across the input.

- Detects a specific pattern (edge, texture...)
- Different kernels → different features
- In CNNs, kernels are **learned** from data

Three kernels — three detectors

Blur

$\frac{1}{9}$	$\frac{1}{9}$	$\frac{1}{9}$
$\frac{1}{9}$	$\frac{1}{9}$	$\frac{1}{9}$
$\frac{1}{9}$	$\frac{1}{9}$	$\frac{1}{9}$

Sobel-X (vertical edges)

↓ we use this

1	0	-1
2	0	-2
1	0	-1

Identity

0	0	0
0	1	0
0	0	0

The Kernel

What is a Kernel?

A small matrix of weights that slides across the input.

- Detects a specific pattern (edge, texture...)
- Different kernels → different features
- In CNNs, kernels are **learned** from data

Three kernels — three detectors

Blur

$\frac{1}{9}$	$\frac{1}{9}$	$\frac{1}{9}$
$\frac{1}{9}$	$\frac{1}{9}$	$\frac{1}{9}$
$\frac{1}{9}$	$\frac{1}{9}$	$\frac{1}{9}$

Sobel-X (vertical edges)

↓ we use this

1	0	-1
2	0	-2
1	0	-1

Identity

0	0	0
0	1	0
0	0	0

In training, the network **learns** its own kernels via **backpropagation** — no hand-design needed

Convolution: Steps

Input (5×5)

3	1	0	2	4
1	5	2	0	3
0	2	4	1	2
3	1	0	3	1
1	0	2	1	4

$\times$

Kernel

1	0	-1
2	0	-2
1	0	-1

$=$

Result

-3

$O[0, 0]$

$\rightarrow$

Output O — building...

-3		

$$3\cdot1 + 1\cdot0 + 0\cdot(-1) + 1\cdot2 + 5\cdot0 + 2\cdot(-2) + 0\cdot1 + 2\cdot0 + 4\cdot(-1) = -3$$

Convolution: Steps

Input (5×5)

3	1	0	2	4
1	5	2	0	3
0	2	4	1	2
3	1	0	3	1
1	0	2	1	4

$\times$

Kernel

1	0	-1
2	0	-2
1	0	-1

$=$

Result

1

$O[0, 1]$

$\rightarrow$

Output O — building...

-3	1	

$$1 \cdot 1 + 0 \cdot 0 + 2 \cdot (-1) + 5 \cdot 2 + 2 \cdot 0 + 0 \cdot (-2) + 2 \cdot 1 + 4 \cdot 0 + 1 \cdot (-1) = 1$$

Convolution: Steps

Input (5×5)

3	1	0	2	4
1	5	2	0	3
0	2	4	1	2
3	1	0	3	1
1	0	2	1	4

$\times$

Kernel

1	0	-1
2	0	-2
1	0	-1

$=$

Result

$$\boxed{6}$$

$O[0, 2]$

$\rightarrow$

Row 0 complete!

-3	1	6

$$0\cdot1 + 2\cdot0 + 4\cdot(-1) + 2\cdot2 + 0\cdot0 + 3\cdot(-2) + 4\cdot1 + 1\cdot0 + 2\cdot(-1) = 6$$

Convolution: Steps

Input (5×5)

3	1	0	2	4
1	5	2	0	3
0	2	4	1	2
3	1	0	3	1
1	0	2	1	4

$\times$

Kernel

1	0	-1
2	0	-2
1	0	-1

$=$

Result

-8

$O[1, 0]$

$\rightarrow$

Output O — building...

-3	1	6
-8		

$$1\cdot1 + 5\cdot0 + 2\cdot(-1) + 0\cdot2 + 2\cdot0 + 4\cdot(-2) + 3\cdot1 + 1\cdot0 + 0\cdot(-1) = -8$$

Convolution: Steps

Input (5×5)

3	1	0	2	4
1	5	2	0	3
0	2	4	1	2
3	1	0	3	1
1	0	2	1	4

$\times$

Kernel

1	0	-1
2	0	-2
1	0	-1

$=$

Result

2

$O[1, 1]$

$\rightarrow$

Output O — **building...**

-3	1	6
-8	2	

$$5 \cdot 1 + 2 \cdot 0 + 0 \cdot (-1) + 2 \cdot 2 + 4 \cdot 0 + 1 \cdot (-2) + 1 \cdot 1 + 0 \cdot 0 + 3 \cdot (-1) = 2$$

Convolution: Steps

Input (5×5)

3	1	0	2	4
1	5	2	0	3
0	2	4	1	2
3	1	0	3	1
1	0	2	1	4

$\times$

Kernel

1	0	-1
2	0	-2
1	0	-1

$=$

Result

6

$O[1, 2]$

$\rightarrow$

Rows 0–1 complete!

-3	1	6
-8	2	6

$$2 \cdot 1 + 0 \cdot 0 + 3 \cdot (-1) + 4 \cdot 2 + 1 \cdot 0 + 2 \cdot (-2) + 0 \cdot 1 + 3 \cdot 0 + 1 \cdot (-1) = 6$$

Convolution: Steps

Input (5×5)

3	1	0	2	4
1	5	2	0	3
0	2	4	1	2
3	1	0	3	1
1	0	2	1	4

$\times$

Kernel

1	0	-1
2	0	-2
1	0	-1

$=$

Result

-4

$O[2, 0]$

$\rightarrow$

Output O — building...

-3	1	6
-8	2	6
-4		

$$0\cdot1 + 2\cdot0 + 4\cdot(-1) + 3\cdot2 + 1\cdot0 + 0\cdot(-2) + 1\cdot1 + 0\cdot0 + 2\cdot(-1) = -4$$

Convolution: Steps

Input (5×5)

3	1	0	2	4
1	5	2	0	3
0	2	4	1	2
3	1	0	3	1
1	0	2	1	4

$\times$

Kernel

1	0	-1
2	0	-2
1	0	-1

$=$

Result

2

$O[2, 1]$

$\rightarrow$

Output O — building...

-3	1	6
-8	2	6
-4	2	

$$2\cdot1 + 4\cdot0 + 1\cdot(-1) + 1\cdot2 + 0\cdot0 + 3\cdot(-2) + 0\cdot1 + 2\cdot0 + 1\cdot(-1) = 2$$

Convolution: Steps

Input (5×5)

3	1	0	2	4
1	5	2	0	3
0	2	4	1	2
3	1	0	3	1
1	0	2	1	4

$\times$

Kernel

1	0	-1
2	0	-2
1	0	-1

$=$

Result

4

$O[2, 2]$

$\rightarrow$

All 9 values computed!

-3	1	6
-8	2	6
-4	2	4

$$4\cdot1 + 1\cdot0 + 2\cdot(-1) + 0\cdot2 + 3\cdot0 + 1\cdot(-2) + 2\cdot1 + 1\cdot0 + 4\cdot(-1) = 4$$

The Feature Map

Output (3×3) — Sobel-X on 5×5 input

-3	1	6
-8	2	6
-4	2	4

Positive — vertical edge found

Negative — opposite edge direction

Near zero — no edge here

The Feature Map

Output (3×3) — Sobel-X on 5×5 input

-3	1	6
-8	2	6
-4	2	4

Positive — vertical edge found

Negative — opposite edge direction

Near zero — no edge here

Output size

$O = N - F + 1$

$(5 - 3 + 1) \times (5 - 3 + 1) = 3 \times 3$ ✓

Multiple kernels

A real CNN uses **32**, **64**, **128**... kernels in parallel — each map detects a different pattern.

ReLU Activation

Definition

$\mathrm{ReLU}(x) = \max(0, x)$

Replaces negatives with **zero**, keeps positives.

Why? Adds non-linearity so stacked layers learn complex patterns — not just one linear transform.

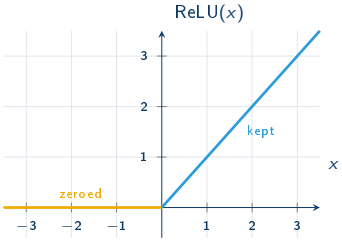
ReLU Activation

Definition

$\mathrm{ReLU}(x) = \max(0, x)$

Replaces negatives with **zero**, keeps positives.

Why? Adds non-linearity so stacked layers learn complex patterns — not just one linear transform.



ReLU: Applied to Feature Map

Before ReLU $\xrightarrow{\max(0,x)}$ **After ReLU**

-3	1	6
-8	2	6
-4	2	4

0	1	6
0	2	6
0	2	4

Negative values (-3, -8, -4) set to 0 • Positive values (1, 6, 2, 6, 2, 4) pass through unchanged

Pooling Layer

What & Why?

Slides a window over the feature map and keeps only the **maximum** value per region.

- Shrinks spatial size (fewer params)
- Retains dominant features
- Adds translation invariance

Output size

$O = \left\lfloor \frac{I-P}{S} \right\rfloor + 1 \quad (I\text{=input},\ P\text{=pool},\ S\text{=stride})$

4×4, pool 2×2, stride 2 $\Rightarrow$ 2×2 output

Pooling Layer

What & Why?

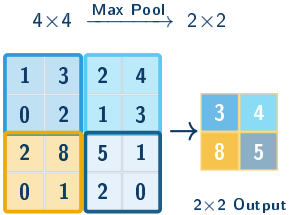
Slides a window over the feature map and keeps only the **maximum** value per region.

- Shrinks spatial size (fewer params)
- Retains dominant features
- Adds translation invariance

Output size

$$O = \left\lfloor \frac{I-P}{S} \right\rfloor + 1 \quad (I\text{=input},\ P\text{=pool},\ S\text{=stride})$$

4×4, pool 2×2, stride 2 $\Rightarrow$ 2×2 output



Max Pooling (2×2, stride 2)

Max Pooling: Step-by-Step

Max Pooling window

Input (4×4)

1	3	2	4
0	2	1	3
2	8	5	1
0	1	2	0

→

Output (2×2)

3	

$\max(1, 3, 0, 2) = 3$

Window (0:2, 0:2) Values: 1, 3, 0, 2 max = 3 ⇒ Output[0, 0] = 3

Pool 2×2, Stride 2 • Input 4×4 → Output 2×2 • 50% size reduction per dimension

Max Pooling: Step-by-Step

Max Pooling window

Input (4×4)

1	3	2	4
0	2	1	3
2	8	5	1
0	1	2	0

$\rightarrow$

Output (2×2)

3	4

$\max(2, 4, 1, 3) = 4$

Window (0:2, 2:4) Values: 2, 4, 1, 3 max = 4 $\Rightarrow$ Output[0, 1] = 4

Max Pooling: Step-by-Step

Max Pooling window

Input (4×4)

1	3	2	4
0	2	1	3
2	8	5	1
0	1	2	0

$\rightarrow$

Output (2×2)

3	4
8	

$\max(2, 8, 0, 1) = 8$

Window (2:4, 0:2) Values: 2, 8, 0, 1 max = 8 $\Rightarrow$ Output[1, 0] = 8

Max Pooling: Step-by-Step

Max Pooling window

Input (4×4)

1	3	2	4
0	2	1	3
2	8	5	1
0	1	2	0

$\max(5, 1, 2, 0) = 5$

$\rightarrow$

Output (2×2)

3	4
8	5

Window (2:4, 2:4) Values: 5, 1, 2, 0 max = 5 $\Rightarrow$ Output: [3, 4; 8, 5] complete!

Pooling: What Did We Achieve?

All 4 windows processed — here is the impact

75%
parameter reduction

2×
compression each dimension

≈ same
dominant features retained

Before Pooling (4×4) After Pooling (2×2)

1	3	2	4
0	2	1	3
2	8	5	1
0	1	2	0

⇒ Max Pool 2×2, s=2

3	4
8	5

Our feature vector is now **compact**. One final step — *interpret* it. ⇒ **Fully Connected Layer**

Pooling: What Did We Achieve?

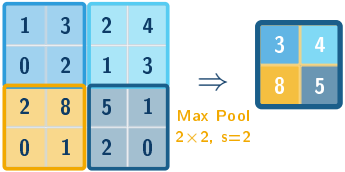
All 4 windows processed — here is the impact

75%
parameter reduction

2×
compression each dimension

≈ same
dominant features retained

Before Pooling (4×4) After Pooling (2×2)



Our feature vector is now **compact**. One final step — *interpret* it. ⇒ **Fully Connected Layer**

Pooling: What Did We Achieve?

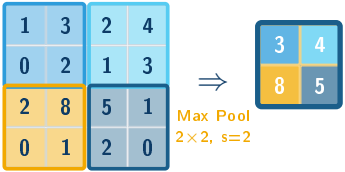
All 4 windows processed — here is the impact

75%
parameter reduction

2×
compression each dimension

≈ same
dominant features retained

Before Pooling (4×4) After Pooling (2×2)



Our feature vector is now **compact**. One final step — *interpret* it. ⇒ **Fully Connected Layer**

Fully Connected Layer

What & Why?

Every neuron connects to **every** neuron in the next layer.

$y = \sigma(Wx + b)$

- **Flatten** pooled map to 1D vector
- **Reason** over all extracted features
- Final FC $\xrightarrow{\text{softmax}}$ class probabilities

Fully Connected Layer

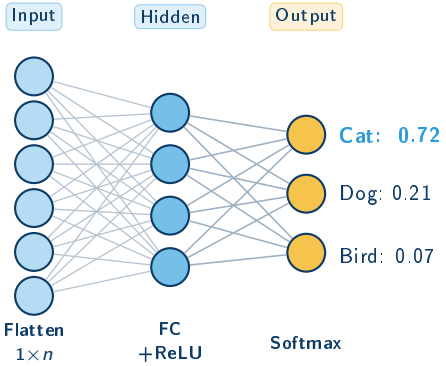
What & Why?

Every neuron connects to **every** neuron in the next layer.

$y = \sigma(Wx + b)$

- **Flatten** pooled map to 1D vector
- **Reason** over all extracted features
- Final FC $\xrightarrow{\text{softmax}}$ class probabilities

Flatten then classify



Putting It All Together

One forward pass — watch the **data shape** shrink at every step

Input Image	Conv Layer	ReLU	Max Pooling	Flatten	FC +ReLU	Softmax Output
$32 \times 32 \times 3$	$30 \times 30 \times 32$	$30 \times 30 \times 32$	$15 \times 15 \times 32$	7200	128	10

Conv extracts

Edges & textures via learned kernels.

ReLU + Pool compress

Non-linearity + spatial reduction.

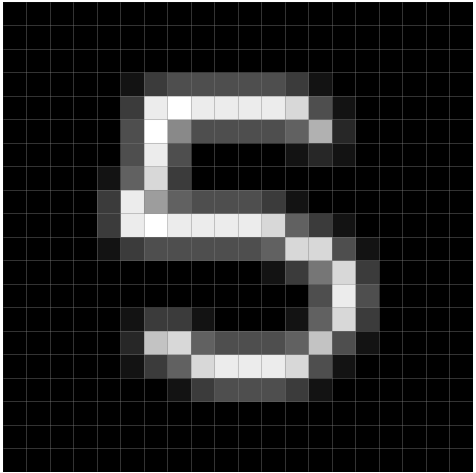
FC classifies

7,200 values $\rightarrow$ 10 class probabilities via softmax.

Let's See a CNN In Action

We'll walk through **layers** of a CNN — from raw pixels to a final **digit prediction**, step by step.

Input Image



20×20 grayscale — colors only

What the CNN Sees

A handwritten digit captured as a 20×20 **grayscale** grid.

Each square is one **pixel**.

Bright $\rightarrow$ pen stroke
Dark $\rightarrow$ background

To the network, this is not a picture — it is a **matrix** of **intensity values**.

Input Image

0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0	0.0	0.0	0.1	0.2	0.2	0.2	0.2	0.2	0.2	0.2	0.1	0.0	0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0	0.0	0.2	0.6	0.7	0.6	0.6	0.6	0.6	0.6	0.2	0.1	0.0	0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0	0.0	0.2	0.7	0.4	0.2	0.2	0.2	0.2	0.2	0.5	0.1	0.0	0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0	0.0	0.2	0.6	0.2	0.0	0.0	0.0	0.0	0.1	0.1	0.1	0.0	0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0	0.1	0.2	0.6	0.2	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0	0.2	0.6	0.4	0.2	0.2	0.2	0.2	0.2	0.1	0.0	0.0	0.0	0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0	0.2	0.6	0.7	0.6	0.6	0.6	0.6	0.6	0.2	0.2	0.1	0.0	0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0	0.1	0.2	0.2	0.2	0.2	0.2	0.2	0.2	0.6	0.6	0.2	0.1	0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.1	0.2	0.3	0.6	0.2	0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.2	0.6	0.2	0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0	0.0	0.1	0.2	0.2	0.1	0.0	0.0	0.0	0.1	0.2	0.6	0.2	0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0	0.0	0.1	0.5	0.6	0.2	0.2	0.2	0.2	0.2	0.5	0.2	0.1	0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0	0.0	0.1	0.2	0.2	0.6	0.6	0.6	0.6	0.6	0.2	0.1	0.0	0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.1	0.2	0.2	0.2	0.2	0.2	0.1	0.0	0.0	0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0

20×20 grayscale — with normalized values

Numerical View

Each pixel is rescaled to the range **[0.0, 1.0]**.

0.0 $\rightarrow$ pure black
1.0 $\rightarrow$ pure white

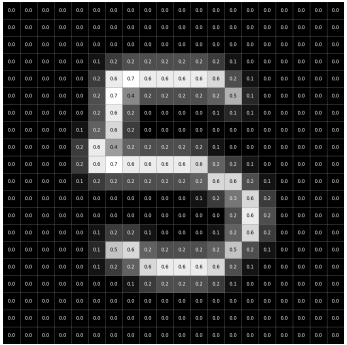
$20 \times 20 = 400$ floats form the network's **input**.

Normalization ensures stable gradients and **faster convergence**.

Convolution — Horizontal Edge Detection

Key Insight

Extracts horizontal edges to identify features like the top bar of a 5.

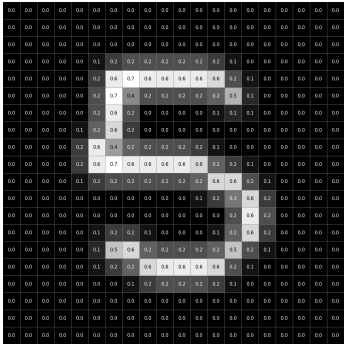


Input 20×20

Convolution — Horizontal Edge Detection

Key Insight

Extracts horizontal edges to identify features like the top bar of a 5.



Input 20×20

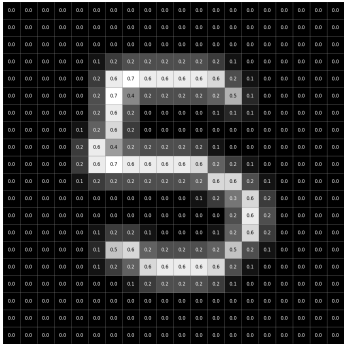
1.0	1.0	1.0
0.0	0.0	0.0
-1.0	-1.0	-1.0

Filter 3×3

Convolution — Horizontal Edge Detection

Key Insight

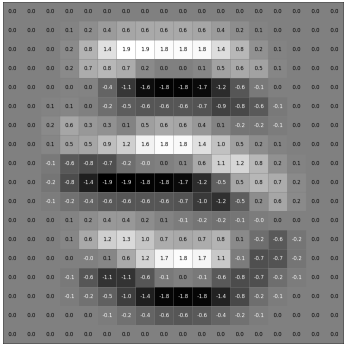
Extracts horizontal edges to identify features like the top bar of a 5.



Input 20×20

Filter 3×3

$*$

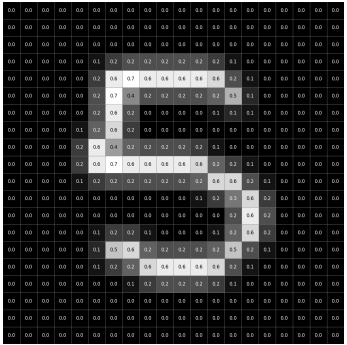


Feature Map 18×18

Convolution — Vertical Edge Detection

Key Insight

Highlights vertical structures to identify features like the straight back of a 1 or stem of a 5.

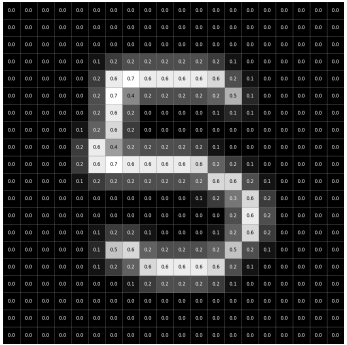


Input 20×20

Convolution — Vertical Edge Detection

Key Insight

Highlights vertical structures to identify features like the straight back of a 1 or stem of a 5.



Input 20×20

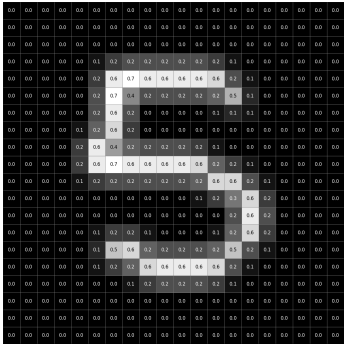
1.0	0.0	-1.0
1.0	0.0	-1.0
1.0	0.0	-1.0

Filter 3×3

Convolution — Vertical Edge Detection

Key Insight

Highlights vertical structures to identify features like the straight back of a 1 or stem of a 5.

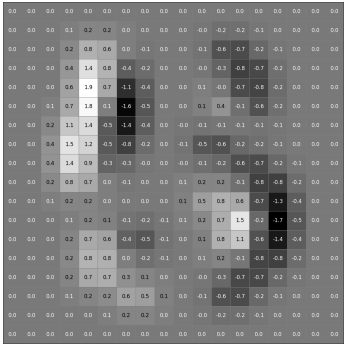


Input 20×20

1.0	0.0	-1.0
1.0	0.0	-1.0
1.0	0.0	-1.0

Filter 3×3

$*$

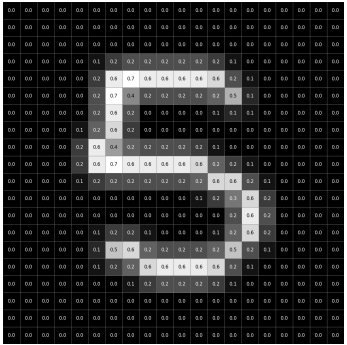


Feature Map 18×18

Convolution — Corner Detection

Key Insight

Detects intersecting lines and sharp bends, crucial for recognizing loops in digits like 8 or 6.

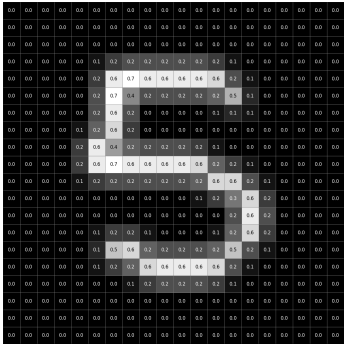


Input 20×20

Convolution — Corner Detection

Key Insight

Detects intersecting lines and sharp bends, crucial for recognizing loops in digits like 8 or 6.



Input 20×20

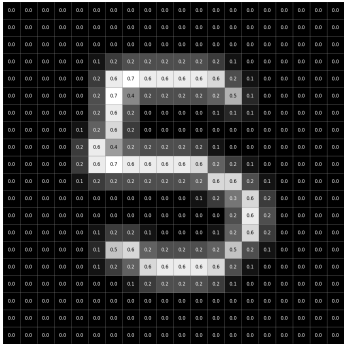
1.0	1.0	0.0
1.0	-1.0	-1.0
0.0	-1.0	-1.0

Filter 3×3

Convolution — Corner Detection

Key Insight

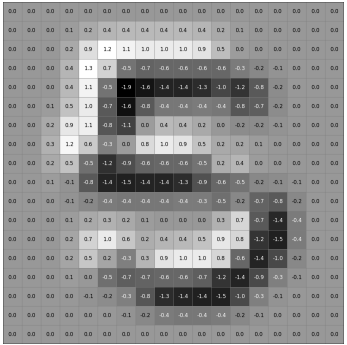
Detects intersecting lines and sharp bends, crucial for recognizing loops in digits like 8 or 6.



Input 20×20

Filter 3×3

$*$

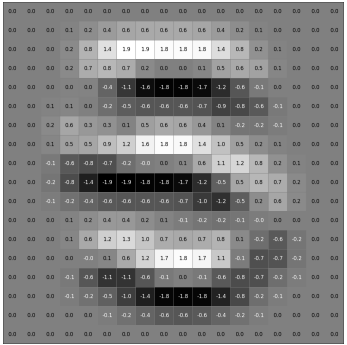


Feature Map 18×18

ReLU — Horizontal Feature Map

Key Insight

Removes negative values (noise) to keep only the most strongly detected horizontal features.

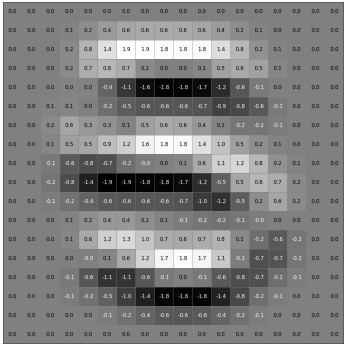


Before ReLU

ReLU — Horizontal Feature Map

Key Insight

Removes negative values (noise) to keep only the most strongly detected horizontal features.



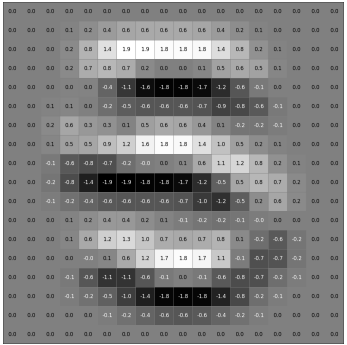
Before ReLU

max(0,x)

ReLU — Horizontal Feature Map

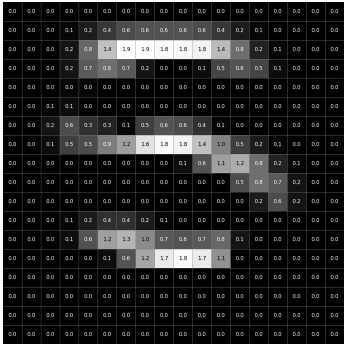
Key Insight

Removes negative values (noise) to keep only the most strongly detected horizontal features.



Before ReLU

max(0,x)

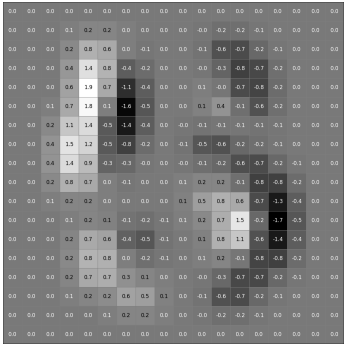


After ReLU

ReLU — Vertical Feature Map

Key Insight

Removes negative values (noise) to keep only the most strongly detected vertical features.

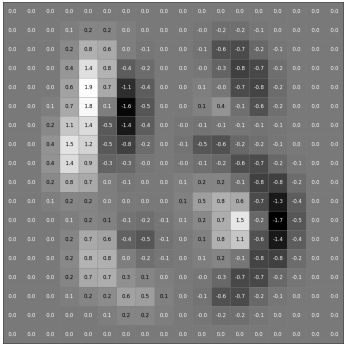


Before ReLU

ReLU — Vertical Feature Map

Key Insight

Removes negative values (noise) to keep only the most strongly detected vertical features.



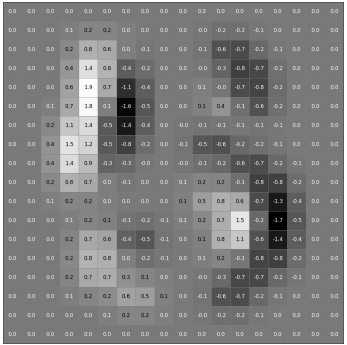
Before ReLU

max(0,x)

ReLU — Vertical Feature Map

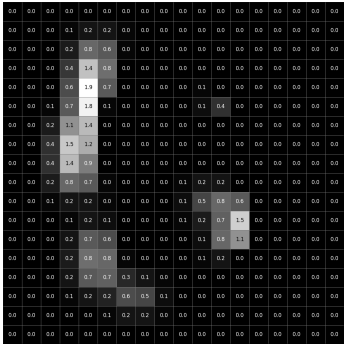
Key Insight

Removes negative values (noise) to keep only the most strongly detected vertical features.



Before ReLU

max(0,x)

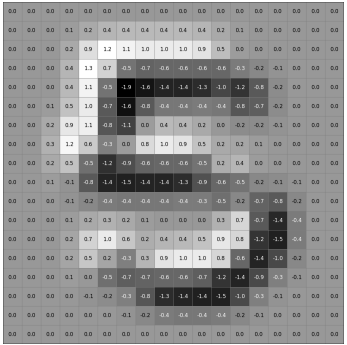


After ReLU

ReLU — Corner Feature Map

Key Insight

Removes negative values (noise) to keep only the most strongly detected corner features.

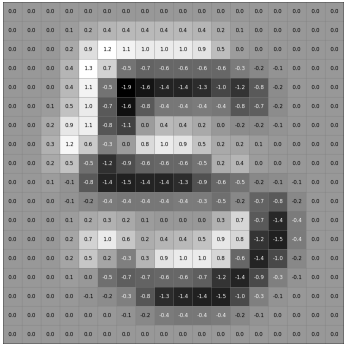


Before ReLU

ReLU — Corner Feature Map

Key Insight

Removes negative values (noise) to keep only the most strongly detected corner features.



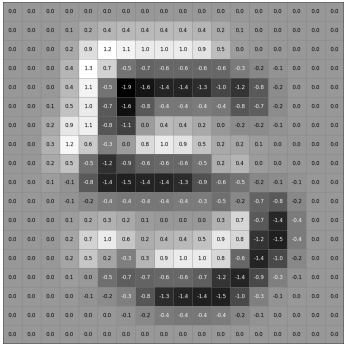
Before ReLU

max(0,x)

ReLU — Corner Feature Map

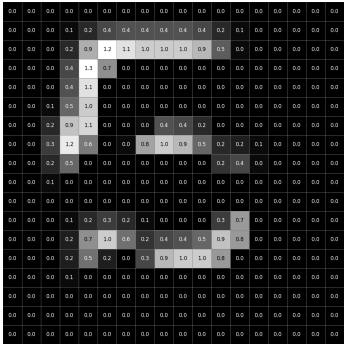
Key Insight

Removes negative values (noise) to keep only the most strongly detected corner features.



Before ReLU

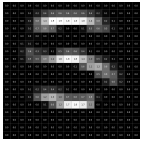
max(0,x)



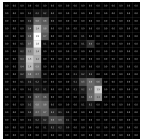
After ReLU

Merging Feature Maps

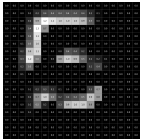
Horizontal



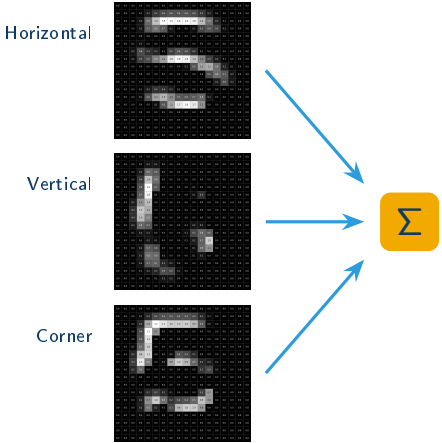
Vertical



Corner

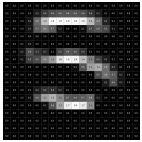


Merging Feature Maps

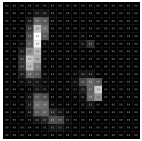


Merging Feature Maps

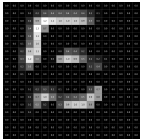
Horizontal



Vertical



Corner



Σ

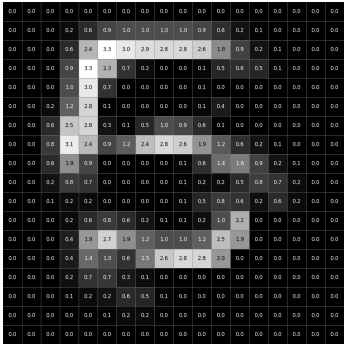
0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.2	0.6	0.9	1.0	1.0	1.0	1.0	0.9	0.6	0.2	0.1	0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.6	2.4	3.3	3.0	2.9	2.8	2.8	2.6	1.8	0.9	0.2	0.1	0.0	0.0	0.0
0.0	0.0	0.0	0.9	3.3	2.3	0.7	0.2	0.0	0.0	0.1	0.5	0.6	0.5	0.1	0.0	0.0	0.0
0.0	0.0	0.0	1.0	3.0	0.7	0.0	0.0	0.0	0.0	0.1	0.0	0.0	0.0	0.0	0.0	0.0	0.0
0.0	0.0	0.2	1.2	2.8	0.1	0.0	0.0	0.0	0.0	0.1	0.4	0.0	0.0	0.0	0.0	0.0	0.0
0.0	0.0	0.6	2.5	2.8	0.3	0.1	0.5	1.0	0.9	0.6	0.1	0.0	0.0	0.0	0.0	0.0	0.0
0.0	0.0	0.8	3.1	2.4	0.9	1.2	2.4	2.8	2.6	1.9	1.2	0.6	0.2	0.1	0.0	0.0	0.0
0.0	0.0	0.6	1.9	0.9	0.0	0.0	0.0	0.0	0.1	0.6	1.4	1.6	0.9	0.2	0.1	0.0	0.0
0.0	0.0	0.2	0.8	0.7	0.0	0.0	0.0	0.0	0.1	0.2	0.2	0.5	0.8	0.7	0.2	0.0	0.0
0.0	0.0	0.1	0.2	0.2	0.0	0.0	0.0	0.0	0.1	0.5	0.8	0.6	0.2	0.6	0.2	0.0	0.0
0.0	0.0	0.0	0.2	0.6	0.8	0.6	0.2	0.1	0.1	0.2	1.0	2.2	0.0	0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.4	1.9	2.7	1.9	1.2	1.0	1.0	1.2	2.5	1.9	0.0	0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.4	1.4	1.0	0.6	1.5	2.6	2.8	2.8	2.0	0.0	0.0	0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.2	0.7	0.7	0.3	0.1	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.1	0.2	0.2	0.6	0.5	0.1	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0	0.0	0.1	0.2	0.2	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0

Merged Feature Map 18×18

Max Pooling and Flattening

Key Insight

Downsamples to keep the most prominent features, then unrolls into a 1D list for the fully connected layer.

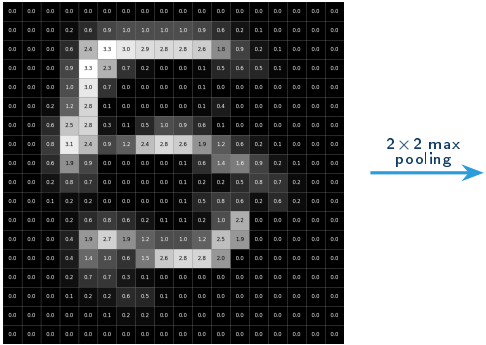


Before Pooling 18×18

Max Pooling and Flattening

Key Insight

Downsamples to keep the most prominent features, then unrolls into a 1D list for the fully connected layer.

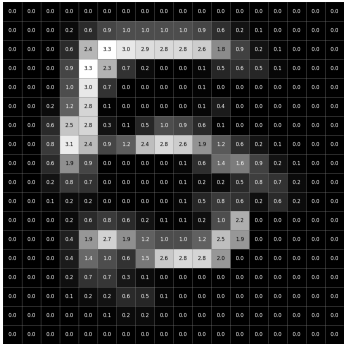


Before Pooling 18×18

Max Pooling and Flattening

Key Insight

Downsamples to keep the most prominent features, then unrolls into a 1D list for the fully connected layer.



Before Pooling 18×18

2×2 max pooling

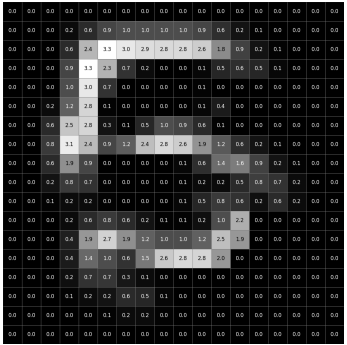
0.0	0.2	0.9	1.0	1.0	0.9	0.2	0.0	0.0
0.0	0.9	3.3	3.0	2.8	2.6	0.9	0.1	0.0
0.0	1.2	3.0	0.0	0.0	0.4	0.0	0.0	0.0
0.0	3.1	2.8	2.4	2.8	1.9	0.6	0.1	0.0
0.0	1.9	0.9	0.0	0.1	1.4	1.6	0.7	0.0
0.0	0.2	0.8	0.6	0.1	1.0	2.2	0.6	0.0
0.0	0.4	2.7	1.9	2.8	2.8	1.9	0.0	0.0
0.0	0.2	0.7	0.6	0.1	0.0	0.0	0.0	0.0
0.0	0.0	0.1	0.2	0.0	0.0	0.0	0.0	0.0

After Pooling 9×9

Max Pooling and Flattening

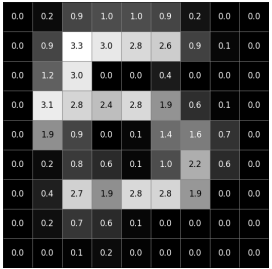
Key Insight

Downsamples to keep the most prominent features, then unrolls into a 1D list for the fully connected layer.



Before Pooling 18×18

2×2 max pooling



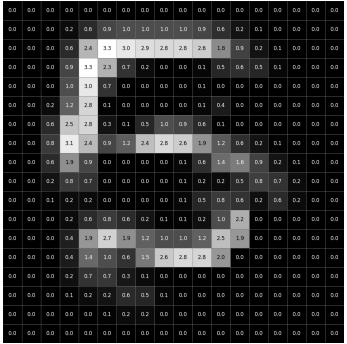
After Pooling 9×9

flatten

Max Pooling and Flattening

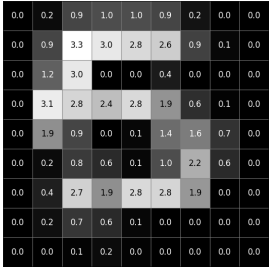
Key Insight

Downsamples to keep the most prominent features, then unrolls into a 1D list for the fully connected layer.



Before Pooling 18×18

2×2 max pooling



After Pooling 9×9

flatten

81×1

Fully Connected Layer and Softmax

Key Insight

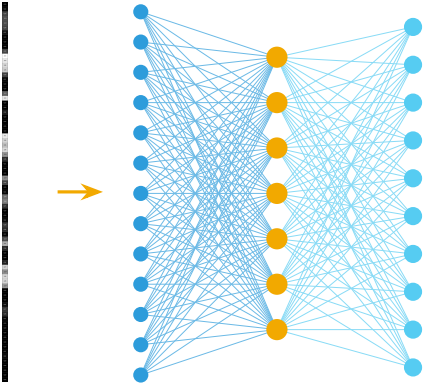
Weighs features to calculate raw scores (logits) for each digit, then converts them to probabilities.

Flatten 81×1

Fully Connected Layer and Softmax

Key Insight

Weighs features to calculate raw scores (logits) for each digit, then converts them to probabilities.

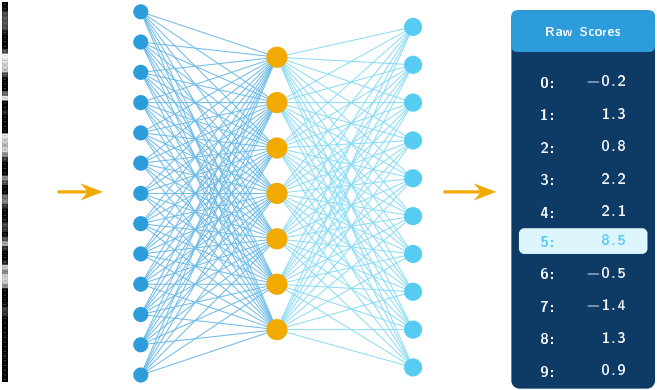


Flatten 81×1

Fully Connected Layer and Softmax

Key Insight

Weighs features to calculate raw scores (logits) for each digit, then converts them to probabilities.

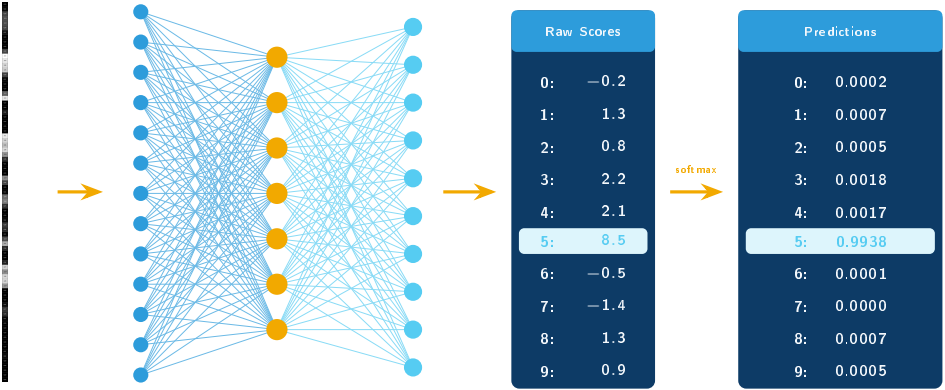


Flatten 81×1

Fully Connected Layer and Softmax

Key Insight

Weighs features to calculate raw scores (logits) for each digit, then converts them to probabilities.



Real-World Impact of CNNs

HEALTHCARE

94.5%

Accuracy in detecting diabetic retinopathy

Google Health, 2016
FDA-approved CNN systems now used in 40+ countries

Real-World Impact of CNNs

HEALTHCARE

94.5%

Accuracy in detecting diabetic retinopathy

Google Health, 2016
FDA-approved CNN systems now used in 40+ countries

AUTONOMOUS VEHICLES

40%

Reduction in accident rate with CNN-based perception

Tesla, Waymo, Cruise
Process 1TB+ sensor data/hr
Real-time object detection

Real-World Impact of CNNs

HEALTHCARE

94.5%

Accuracy in detecting diabetic retinopathy

Google Health, 2016
FDA-approved CNN systems now used in 40+ countries

AUTONOMOUS VEHICLES

40%

Reduction in accident rate with CNN-based perception

Tesla, Waymo, Cruise
Process 1TB+ sensor data/hr
Real-time object detection

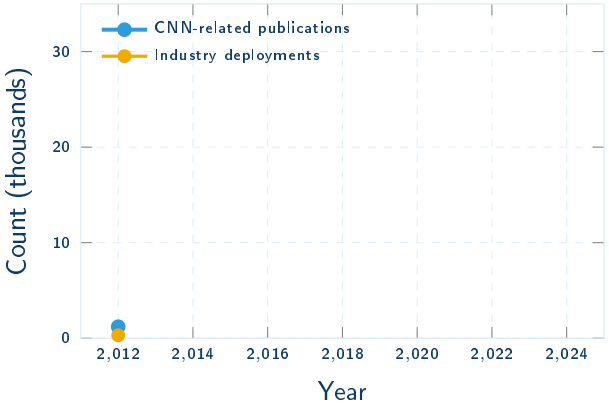
SATELLITE & AGRICULTURE

89%

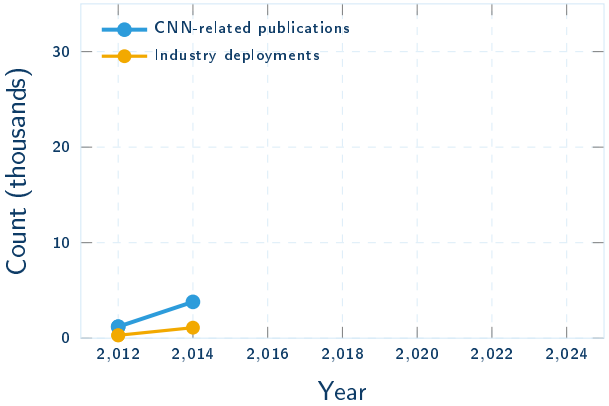
Crop disease detection accuracy from aerial images

PlantVillage dataset
Deforestation monitoring
Yield prediction $\pm 5\%$

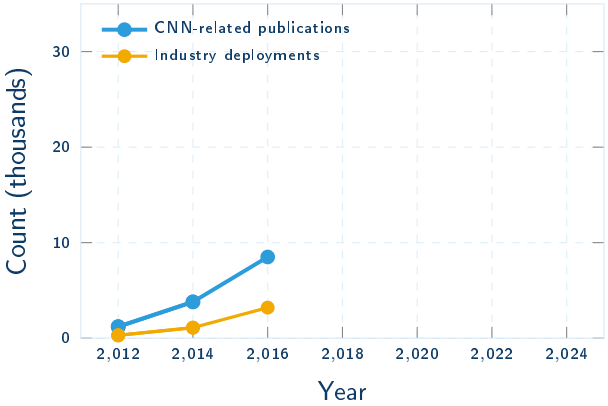
CNN Adoption: Growth Over Time



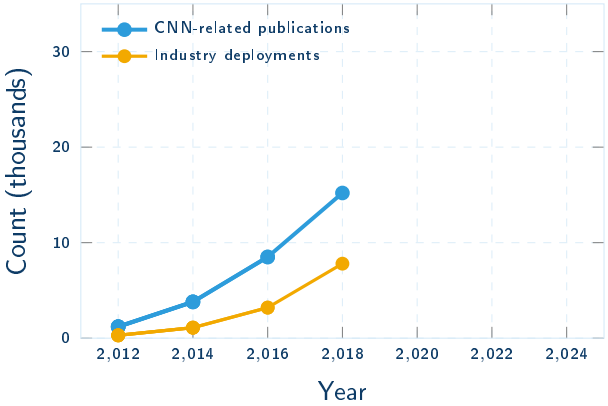
CNN Adoption: Growth Over Time



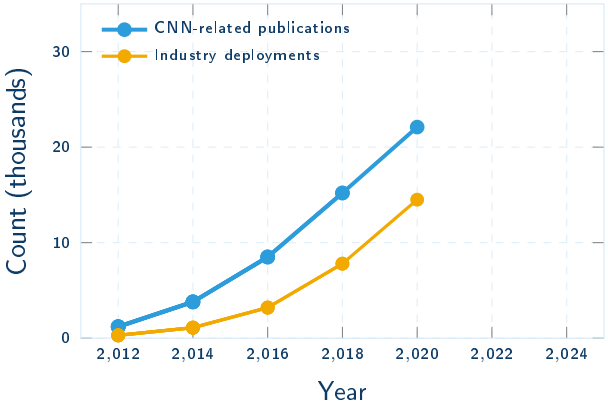
CNN Adoption: Growth Over Time



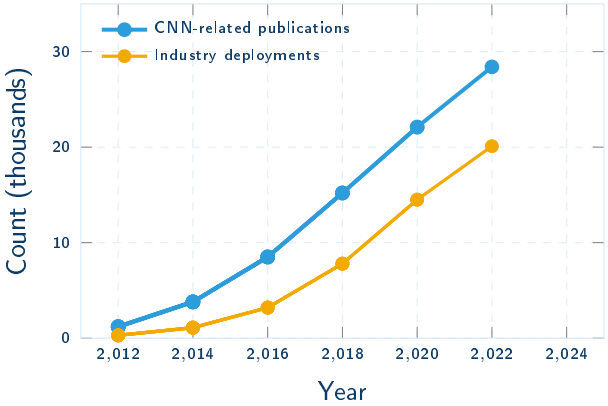
CNN Adoption: Growth Over Time



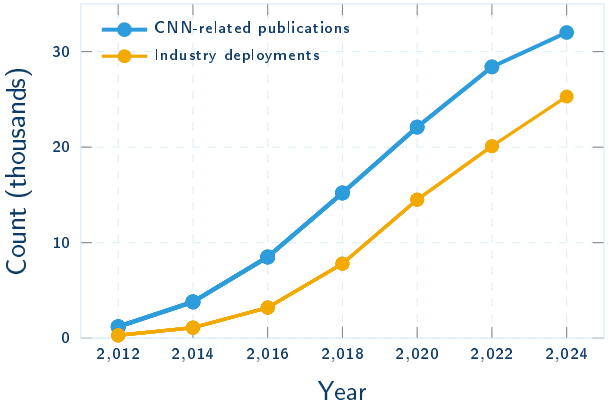
CNN Adoption: Growth Over Time



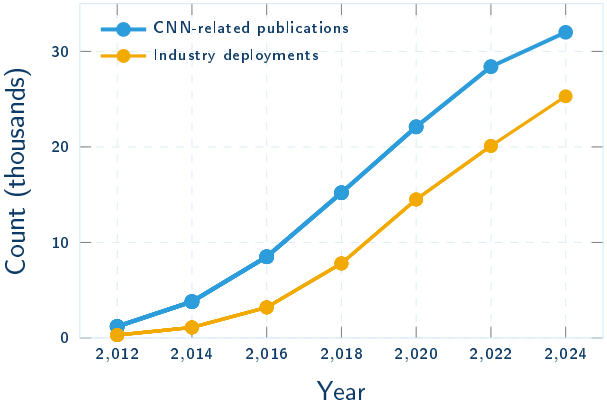
CNN Adoption: Growth Over Time



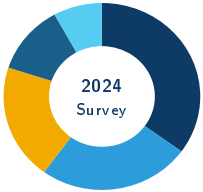
CNN Adoption: Growth Over Time



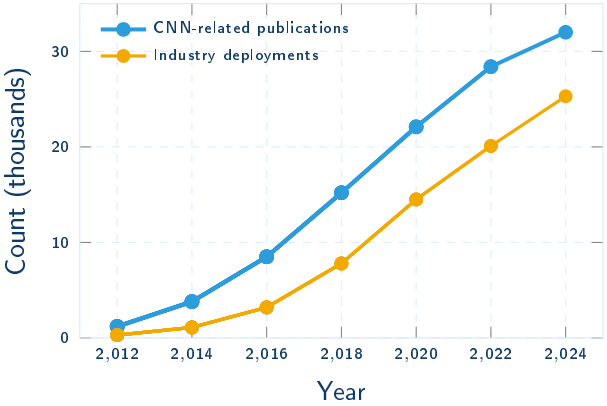
CNN Adoption: Growth Over Time



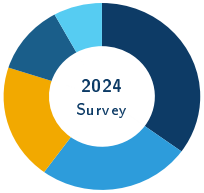
Architecture Usage



CNN Adoption: Growth Over Time



Architecture Usage



- ResNet — 35%
- EfficientNet — 25%
- YOLO — 20%
- VGG — 12%
- Other — 8%

Limitations of CNNs

COMPUTATIONAL COST

300W+

GPU power consumption during training

GPT-scale models: $10M+ training cost; ResNet-152: 60M parameters; days on 8×A100 GPUs

Limitations of CNNs

COMPUTATIONAL COST

300W+

GPU power consumption during training

GPT-scale models: $10M+ training cost; ResNet-152: 60M parameters; days on 8×A100 GPUs

DATA HUNGER

1M+

Labeled images needed for robust training

ImageNet: 14M images
Medical data: scarce & expensive
Labeling cost: $0.05–$6/image
Bias amplification risk

Limitations of CNNs

COMPUTATIONAL COST

300W+

GPU power consumption during training

GPT-scale models: $10M+ training cost; ResNet-152: 60M parameters; days on 8×A100 GPUs

DATA HUNGER

1M+

Labeled images needed for robust training

ImageNet: 14M images
Medical data: scarce & expensive
Labeling cost: $0.05–$6/image
Bias amplification risk

BLACK BOX PROBLEM

87%

Of practitioners report difficulty explaining CNN decisions to stakeholders

EU AI Act: explainability required for high-risk systems
Grad-CAM helps but insufficient for regulation

Thank You

ID: 2205031, 2205046, 2205059